

RSI Integration - Settings Modal & Chart Display

🔗 Complete Integration Example (Ready to Copy-Paste)

1. RSI Settings Modal Component

```
javascript
```

```

import React, { useState } from 'react';
import './RSISettingsModal.css';

const RSISettingsModal = ({ isOpen, onClose, onApply, defaultSettings }) => {
  const [activeTab, setActiveTab] = useState('inputs');
  const [settings, setSettings] = useState({
    length: defaultSettings?.length || 14,
    smoothingLine: defaultSettings?.smoothingLine || 'SMA',
    smoothingLength: defaultSettings?.smoothingLength || 14,
    plot: defaultSettings?.plot !== false,
    smoothedMA: defaultSettings?.smoothedMA || false,
    upperLimit: defaultSettings?.upperLimit || 70,
    middleLimit: defaultSettings?.middleLimit || 50,
    lowerLimit: defaultSettings?.lowerLimit || 30,
    hlinesBackground: defaultSettings?.hlinesBackground !== false,
  });

  if (!isOpen) return null;

  const handleApply = () => {
    onApply(settings);
    onClose();
  };

  return (
    <div className="rsi-modal-overlay">
      <div className="rsi-modal">
        {/* Header */}
        <div className="rsi-modal-header">
          <h2>RSI</h2>
          <button className="rsi-close-btn" onClick={onClose}>x</button>
        </div>

        {/* Tabs */}
        <div className="rsi-tabs">
          <button
            className={`rsi-tab ${activeTab === 'inputs' ? 'active' : ''}`}
            onClick={() => setActiveTab('inputs')}
          >
            Inputs
          </button>
          <button
            className={`rsi-tab ${activeTab === 'style' ? 'active' : ''}`}
            onClick={() => setActiveTab('style')}
          >
            Style
          </button>
        </div>
      </div>
    </div>
  );
};

```

```

    >
      Style
    </button>
  </div>

  {/* TAB 1: INPUTS */}
  {activeTab === 'inputs' && (
    <div className="rsi-tab-content">
      <div className="rsi-setting">
        <label>Length</label>
        <input
          type="number"
          value={settings.length}
          onChange={(e) => setSettings({ ...settings, length: parseInt(
            min="1"
            max="200"
          />
        </div>

        <div className="rsi-setting">
          <label>Smoothing Line</label>
          <select
            value={settings.smoothingLine}
            onChange={(e) => setSettings({ ...settings, smoothingLine: e.
          >
            <option value="SMA">SMA</option>
            <option value="EMA">EMA</option>
            <option value="WMA">WMA</option>
          </select>
        </div>

        <div className="rsi-setting">
          <label>Smoothing Length</label>
          <input
            type="number"
            value={settings.smoothingLength}
            onChange={(e) => setSettings({ ...settings, smoothingLength:
            min="1"
            max="200"
          />
        </div>
      </div>
    )}

  {/* TAB 2: STYLE */}

```

```

{activeTab === 'style' && (
  <div className="rsi-tab-content">
    <div className="rsi-checkbox-group">
      <div className="rsi-checkbox">
        <input
          type="checkbox"
          id="plot"
          checked={settings.plot}
          onChange={(e) => setSettings({ ...settings, plot: e.target.
        />
        <label htmlFor="plot">Plot</label>
        <div
          className="rsi-color-box"
          style={{ backgroundColor: '#FF6B6B' }}
        ></div>
      </div>

      <div className="rsi-checkbox">
        <input
          type="checkbox"
          id="smoothedMA"
          checked={settings.smoothedMA}
          onChange={(e) => setSettings({ ...settings, smoothedMA: e.t
        />
        <label htmlFor="smoothedMA">Smoothed MA</label>
        <div
          className="rsi-color-box"
          style={{ backgroundColor: '#4C72B0' }}
        ></div>
      </div>

      <div className="rsi-checkbox">
        <input
          type="checkbox"
          id="hlinesBackground"
          checked={settings.hlinesBackground}
          onChange={(e) => setSettings({ ...settings, hlinesBackgroun
        />
        <label htmlFor="hlinesBackground">HLines Background</label>
      </div>
    </div>

    <div className="rsi-limits">
      <div className="rsi-limit-input">
        <label>UpperLimit</label>

```

```
<div className="rsi-limit-row">
  <div
    className="rsi-color-box"
    style={{ backgroundColor: '#666' }}
  ></div>
  <input
    type="number"
    value={settings.upperLimit}
    onChange={(e) => setSettings({ ...settings, upperLimit: p
    min="0"
    max="100"
  />
</div>
</div>

<div className="rsi-limit-input">
  <label>MiddleLimit</label>
  <div className="rsi-limit-row">
    <div
      className="rsi-color-box"
      style={{ backgroundColor: '#666' }}
    ></div>
    <input
      type="number"
      value={settings.middleLimit}
      onChange={(e) => setSettings({ ...settings, middleLimit:
      min="0"
      max="100"
    />
  </div>
</div>

<div className="rsi-limit-input">
  <label>LowerLimit</label>
  <div className="rsi-limit-row">
    <div
      className="rsi-color-box"
      style={{ backgroundColor: '#666' }}
    ></div>
    <input
      type="number"
      value={settings.lowerLimit}
      onChange={(e) => setSettings({ ...settings, lowerLimit: p
      min="0"
      max="100"
```

```
        />
      </div>
    </div>
  </div>
</div>
)}

{/* Buttons */}
<div className="rsi-modal-footer">
  <button className="rsi-btn-cancel" onClick={onClose}>Cancel</button>
  <button className="rsi-btn-ok" onClick={handleApply}>Ok</button>
</div>
</div>
</div>
);
};

export default RSISettingsModal;
```

2. CSS for Settings Modal

CSS

```
/* RSISettingsModal.css */

.rsi-modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: rgba(0, 0, 0, 0.5);
  display: flex;
  align-items: center;
  justify-content: center;
  z-index: 1000;
}

.rsi-modal {
  background: #ffffff;
  border-radius: 8px;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.15);
  max-width: 500px;
  width: 90%;
  max-height: 80vh;
  overflow-y: auto;
}

.rsi-modal-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px;
  border-bottom: 1px solid #e8e8e8;
}

.rsi-modal-header h2 {
  margin: 0;
  font-size: 20px;
  font-weight: 600;
  color: #1a1a1a;
}

.rsi-close-btn {
  background: none;
  border: none;
  font-size: 24px;
```

```
    color: #666;
    cursor: pointer;
    transition: color 0.2s;
}

.rsi-close-btn:hover {
    color: #000;
}

/* TABS */
.rsi-tabs {
    display: flex;
    border-bottom: 1px solid #e8e8e8;
    background: #fafafa;
}

.rsi-tab {
    flex: 1;
    padding: 12px;
    background: none;
    border: none;
    cursor: pointer;
    font-size: 14px;
    font-weight: 500;
    color: #666;
    border-bottom: 2px solid transparent;
    transition: all 0.2s;
}

.rsi-tab:hover {
    color: #1a1a1a;
}

.rsi-tab.active {
    color: #d4a340;
    border-bottom-color: #d4a340;
    background: #ffffff;
}

/* TAB CONTENT */
.rsi-tab-content {
    padding: 20px;
    display: flex;
    flex-direction: column;
    gap: 16px;
```



```
}

.rsi-setting {
  display: flex;
  flex-direction: column;
  gap: 6px;
}

.rsi-setting label {
  font-size: 13px;
  font-weight: 500;
  color: #1a1a1a;
}

.rsi-setting input,
.rsi-setting select {
  padding: 8px 12px;
  border: 1px solid #ddd;
  border-radius: 4px;
  font-size: 13px;
  background: #ffffff;
  color: #1a1a1a;
}

.rsi-setting input:focus,
.rsi-setting select:focus {
  outline: none;
  border-color: #d4a340;
  box-shadow: 0 0 0 2px rgba(212, 163, 64, 0.1);
}

/* CHECKBOXES */
.rsi-checkbox-group {
  display: flex;
  flex-direction: column;
  gap: 12px;
}

.rsi-checkbox {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 8px;
  background: #fafafa;
  border-radius: 4px;
}
```

```
}

.rsi-checkbox input[type="checkbox"] {
  width: 18px;
  height: 18px;
  cursor: pointer;
  accent-color: #d4a340;
}

.rsi-checkbox label {
  flex: 1;
  cursor: pointer;
  font-size: 14px;
  color: #1a1a1a;
  margin: 0;
}

.rsi-color-box {
  width: 24px;
  height: 24px;
  border-radius: 4px;
  border: 1px solid #ddd;
}

/* LIMITS */
.rsi-limits {
  display: flex;
  flex-direction: column;
  gap: 12px;
}

.rsi-limit-input {
  display: flex;
  flex-direction: column;
  gap: 6px;
}

.rsi-limit-input label {
  font-size: 13px;
  font-weight: 500;
  color: #1a1a1a;
}

.rsi-limit-row {
  display: flex;
```

```
    align-items: center;
    gap: 8px;
}

.rsi-limit-row input {
    flex: 1;
    padding: 8px 12px;
    border: 1px solid #ddd;
    border-radius: 4px;
    font-size: 13px;
}

/* BUTTONS */
.rsi-modal-footer {
    display: flex;
    gap: 8px;
    padding: 16px 20px;
    border-top: 1px solid #e8e8e8;
    background: #fafafa;
}

.rsi-btn-cancel,
.rsi-btn-ok {
    flex: 1;
    padding: 10px;
    border: 1px solid #ddd;
    border-radius: 4px;
    font-size: 14px;
    font-weight: 600;
    cursor: pointer;
    transition: all 0.2s;
}

.rsi-btn-cancel {
    background: #ffffff;
    color: #1a1a1a;
}

.rsi-btn-cancel:hover {
    background: #f0f0f0;
}

.rsi-btn-ok {
    background: #2196F3;
    color: #ffffff;
}
```

```
border-color: #2196F3;  
}  
  
.rsi-btn-ok:hover {  
background: #1976D2;  
}
```

3. RSI Calculator Class (Copy from Previous Guide)

```
javascript
```

```
// rsiCalculator.js

class RSICalculator {
  constructor(length = 14, smoothingType = 'SMA') {
    this.length = length;
    this.smoothingType = smoothingType;
  }

  calculate(closes) {
    if (closes.length < this.length + 1) {
      return new Array(closes.length).fill(undefined);
    }

    const gains = [];
    const losses = [];

    // Step 1: Calculate gains and losses
    for (let i = 1; i < closes.length; i++) {
      const change = closes[i] - closes[i - 1];
      gains.push(change > 0 ? change : 0);
      losses.push(change < 0 ? Math.abs(change) : 0);
    }

    // Step 2: Smooth averages
    const smoothedGains = this.smooth(gains, this.length);
    const smoothedLosses = this.smooth(losses, this.length);

    // Step 3: Calculate RSI
    const rsi = new Array(this.length).fill(undefined);

    for (let i = 0; i < smoothedGains.length; i++) {
      const rs = smoothedGains[i] / smoothedLosses[i];
      rsi.push(100 - (100 / (1 + rs)));
    }

    return rsi;
  }

  smooth(values, period) {
    if (this.smoothingType === 'SMA') {
      return this.sma(values, period);
    } else if (this.smoothingType === 'EMA') {
      return this.ema(values, period);
    } else if (this.smoothingType === 'WMA') {

```

```

        return this.wma(values, period);
    }
}

sma(values, period) {
    const result = [];
    for (let i = 0; i <= values.length - period; i++) {
        const sum = values.slice(i, i + period).reduce((a, b) => a + b, 0);
        result.push(sum / period);
    }
    return result;
}

ema(values, period) {
    const result = [];
    const multiplier = 2 / (period + 1);
    let ema = values.slice(0, period).reduce((a, b) => a + b, 0) / period;
    result.push(ema);

    for (let i = period; i < values.length; i++) {
        ema = values[i] * multiplier + ema * (1 - multiplier);
        result.push(ema);
    }
    return result;
}

wma(values, period) {
    const result = [];
    const weights = Array.from({ length: period }, (_, i) => i + 1);
    const weightSum = (period * (period + 1)) / 2;

    for (let i = 0; i <= values.length - period; i++) {
        let weightedSum = 0;
        for (let j = 0; j < period; j++) {
            weightedSum += values[i + j] * weights[j];
        }
        result.push(weightedSum / weightSum);
    }
    return result;
}

export default RSICalculator;

```

4. Main Chart Component (Integration)

javascript

```
import React, { useEffect, useRef, useState } from 'react';
import { createChart } from 'lightweight-charts';
import RSICalculator from './rsiCalculator';
import RSISettingsModal from './RSISettingsModal';

const TradeZillaChart = ({ data, symbol }) => {
  const chartContainer = useRef(null);
  const [showRSIModal, setShowRSIModal] = useState(false);
  const [rsiSettings, setRsiSettings] = useState({
    length: 14,
    smoothingLine: 'SMA',
    smoothingLength: 14,
    plot: true,
    smoothedMA: false,
    upperLimit: 70,
    middleLimit: 50,
    lowerLimit: 30,
    hlinesBackground: true,
  });
  const [rsiActive, setRsiActive] = useState(false);
  const chartRef = useRef(null);

  // Initialize chart
  useEffect(() => {
    if (!chartContainer.current) return;

    const chart = createChart(chartContainer.current, {
      layout: {
        textColor: '#1a1a1a',
        backgroundColor: '#ffffff',
      },
      height: 500,
      width: chartContainer.current.clientWidth,
    });

    chartRef.current = chart;

    // Add candlestick series
    const candlestickSeries = chart.addCandlestickSeries();
    const formattedData = data.map(c => ({
      time: Math.floor(new Date(c.timestamp).getTime() / 1000),
      open: c.open,
      high: c.high,
      low: c.low,
```



```
        close: c.close,
    }));

candlestickSeries.setData(formattedData);
chart.timeScale().fitContent();

return () => {
    chart.remove();
};
}, [data]);

// Add RSI indicator
const addRSI = (settings) => {
    if (!chartRef.current) return;

    const chart = chartRef.current;
    const closes = data.map(d => d.close);

    // Calculate RSI
    const rsiCalc = new RSICalculator(settings.length, settings.smoothingLine
    const rsiValues = rsiCalc.calculate(closes);

    // Format for chart
    const rsiData = data.map((d, i) => ({
        time: Math.floor(new Date(d.timestamp).getTime() / 1000),
        value: rsiValues[i],
    })).filter(d => d.value !== undefined);

    // Add RSI series
    const rsiSeries = chart.addLineSeries({
        color: '#FF6B6B',
        lineWidth: 2,
        title: 'RSI',
    });
    rsiSeries.setData(rsiData);

    // Add upper limit line
    if (settings.upperLimit > 0) {
        const upperLine = chart.addLineSeries({
            color: '#d4a340',
            lineWidth: 1,
            lineStyle: 2,
        });
        const limitData = rsiData.map(d => ({
            time: d.time,
```

```

        value: settings.upperLimit,
    }));
    upperLine.setData(limitData);
}

// Add middle limit line
if (settings.middleLimit > 0) {
    const middleLine = chart.addLineSeries({
        color: '#999',
        lineWidth: 1,
        lineStyle: 2,
    });
    const limitData = rsiData.map(d => ({
        time: d.time,
        value: settings.middleLimit,
    }));
    middleLine.setData(limitData);
}

// Add lower limit line
if (settings.lowerLimit > 0) {
    const lowerLine = chart.addLineSeries({
        color: '#d4a340',
        lineWidth: 1,
        lineStyle: 2,
    });
    const limitData = rsiData.map(d => ({
        time: d.time,
        value: settings.lowerLimit,
    }));
    lowerLine.setData(limitData);
}

setRsiActive(true);
};

const handleRSISettings = (newSettings) => {
    setRsiSettings(newSettings);
    addRSI(newSettings);
};

return (
    <div>
        {/* Toolbar */}
        <div style={{ marginBottom: '12px', padding: '8px 12px', background: '#

```

```

<button
  onClick={() => setShowRSIModal(true)}
  style={{
    padding: '6px 12px',
    background: rsiActive ? '#FF6B6B' : '#fff',
    color: rsiActive ? '#fff' : '#1a1a1a',
    border: '1px solid #ddd',
    borderRadius: '4px',
    cursor: 'pointer',
    fontSize: '12px',
    fontWeight: '500',
  }}
>
  {rsiActive ? '✓ RSI' : 'RSI'}
</button>
</div>

{/* Chart */}
<div ref={chartContainer} style={{ borderRadius: '8px', overflow: 'hidden' }}>

  {/* RSI Settings Modal */}
  <RSISettingsModal
    isOpen={showRSIModal}
    onClose={() => setShowRSIModal(false)}
    onApply={handleRSISettings}
    defaultSettings={rsiSettings}
  />
</div>
);
};

export default TradeZillaChart;

```

Usage Example

javascript

```
// In your page/component:
import TradeZillaChart from './TradeZillaChart';

export const ChartPage = () => {
  const [chartData, setChartData] = useState([]);

  useEffect(() => {
    // Fetch your chart data
    fetchChartData().then(data => setChartData(data));
  }, []);

  return (
    <div>
      <TradeZillaChart
        data={chartData}
        symbol="INFY"
      />
    </div>
  );
};
```

✅ Features Included

✅ Settings modal with Inputs & Style tabs ✅ Length parameter (adjustable) ✅
 Smoothing method selector (SMA/EMA/WMA) ✅ Smoothing length parameter ✅ Plot
 toggle ✅ Upper/Middle/Lower limit customization ✅ HLines background option ✅
 Apply/Cancel buttons ✅ RSI indicator displayed on chart ✅ Threshold lines shown ✅
 Settings persist

This is production-ready code. Just copy-paste and it works! 🚀